

UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO
CIÊNCIA DA COMPUTAÇÃO

GRUPO 1

JONATHAN ALVES BISPO DA PAZ
MATRÍCULA: 2024200497

LEANDRO BROGNOLI GRAZZIOTIN
MATRÍCULA: 2024200523

VICTOR DA ROCHA TONIATO
MATRÍCULA: 2024200493

COMPACTAÇÃO DE ARQUIVOS
ESTRUTURA DE DADOS II - 2026/01

LISTA DE FIGURAS

Figura 1 - Representação da estrutura do Heap.....	6
Figura 2 - Função mallocSafe() implementada em heap.c.....	7
Figura 3 - Função troca() implementada em heap.c.....	7
Figura 4 - Função criaHeap() implementada em heap.c.....	8
Figura 5 - Função heapifyUp() implementada em heap.c.....	8
Figura 6 - Função heapifyDown() implementada em heap.c.....	9
Figura 7 - Função insereHeap() implementada em heap.c.....	10
Figura 8 - Função extraiMin() implementada em heap.c.....	10
Figura 9 - Função liberaHeap() implementada em heap.c.....	11
Figura 10 - Estrutura NoHuffman implementada em Huffman.c.....	11
Figura 11 - Função ehMenorNo() implementada em huffman.c.....	12
Figura 12 - Função verificaArquivo() implementada em huffman.c.....	13
Figura 13 - Função formataNome() implementada em huffman.c.....	13
Figura 14 - Função criaArquivoSaida() implementada em huffman.c.....	14
Figura 15 - Função contaFrequencia() implementada em huffman.c.....	14
Figura 16 - Função imprimeFrequencia() implementada em huffman.c.....	15
Figura 17 - Função criaNoHuffman() implementada em huffman.c.....	16
Figura 18 - Função constroiArvore() implementada em huffman.c.....	16
Figura 19 - Função criaTabelaCodigos() implementada em huffman.c.....	17
Figura 20 - Função imprimeArvore() implementada em huffman.c.....	18
Figura 21 - Função comprimeArquivo() implementada em huffman.c.....	19
Figura 22 - Continuação da função comprimeArquivo().....	20
Figura 23 - Final da função comprimeArquivo().....	21
Figura 24 - Função descomprimeArquivo() implementada em huffman.c.....	22
Figura 25 - Continuação da função descomprimeArquivo().....	22
Figura 26 - Final da descomprimeArquivo().....	23
Figura 27 - Função liberaArvore() implementada em huffman.c.....	24

Figura 28 - Início da função main() implementada em main.c.....	25
Figura 29 - Loop do menu implementado.....	25
Figura 30 - Case 1 para compressão de arquivos.....	26
Figura 31 - Case 2 para impressão da tabela de frequências.....	27
Figura 32 - Case 3 para impressão da árvore.....	27
Figuras 33.1 e 33.2 - Case 4 para descompressão de arquivos.....	28
Figura 34 - Case 5 para sair do programa.....	28
Figura 35 - Comandos para executar o programa.....	29
Figura 36 - Representação do menu no terminal.....	29
Figura 37 - Representação do menu no terminal.....	30
Figura 38 - Representação saída da opção 2 no terminal.....	30
Figura 39 - Representação saída da opção 3 no terminal.....	30
Figura 40 - Representação da saída da opção 4 no terminal.....	31
Figura 41 - Encerramento do programa.....	31
Figura 42 - Comando para comparação de arquivos.....	31
Figura 43 - Comparação dos arquivos no Explorador de Arquivos.....	32
Figura 44 - Comparação dos arquivos no Explorador de Arquivos.....	32
Figura 45 - Comparação dos arquivos no Explorador de Arquivos.....	32
Figura 46 - Comparação dos arquivos no Explorador de Arquivos.....	33

SUMÁRIO

1 INTRODUÇÃO.....	4
2 FILA DE PRIORIDADES PARA TIPOS GENÉRICOS DE DADOS.....	5
2.1 Funções complementares implementadas em heap.c.....	5
2.1.1 mallocSafe().....	5
2.1.2 troca().....	6
2.2 Funções principais implementadas em heap.c.....	7
2.2.1 criaHeap().....	7
2.2.2 heapifyUp().....	7
2.2.3 heapifyDown().....	8
2.2.4 insereHeap().....	8
2.2.5 extraiMin().....	9
2.2.6 liberaHeap().....	10
3 IMPLEMENTAÇÃO DA COMPACTAÇÃO DE ARQUIVOS DE TEXTO.....	10
3.1 Funções complementares implementadas em huffman.c.....	11
3.1.1 ehMenorNo().....	11
3.1.2 verificaArquivo().....	12
3.1.3 formataNome().....	12
3.1.4 criaArquivoSaida().....	13
3.2 Funções principais implementadas em huffman.c.....	13
3.2.1 contaFrequencia().....	13
3.2.2 imprimeFrequencia().....	14
3.2.3 criaNoHuffman().....	14
3.2.4 constroiArvore().....	15
3.2.5 criaTabelaCodigos().....	16
3.2.6 imprimeArvore().....	18
3.2.7 comprimeArquivo().....	19

3.2.8 descomprimeArquivo().....	21
3.2.9 liberaArvore().....	24
4 MENU IMPLEMENTADO EM MAIN.C.....	24
5 RESULTADOS E EXEMPLOS.....	29
REFERÊNCIAS.....	33

1 INTRODUÇÃO

A codificação de *Huffman* é um método de compressão que usa as probabilidades de ocorrência dos símbolos no conjunto de dados a ser comprimido para determinar códigos de tamanho variável para cada símbolo através de uma abordagem *bottom-up* (de baixo para cima). No presente trabalho, buscamos desenvolver um programa para a compactação e descompactação de arquivos com o auxílio da fila de prioridade para tratamento de dados genéricos.

O trabalho foi dividido em três etapas principais: a implementação do **heap.c** para filas de prioridade e tratamento de dados genéricos, implementação do **huffman.c** para as funcionalidades exigidas, e logo após, a criação de um menu interativo no arquivo **main.c** com as seguintes opções:

- 1 - Comprimir um arquivo
- 2 - Contagem de ocorrências de elementos no arquivo de texto
- 3 - Imprimir o Código de *Huffman* para o arquivo de texto
- 4 - Descomprimir um arquivo
- 5 - Sair

Vale ressaltar que as opções 2 e 3 dependem de que o usuário tenha previamente escolhido a opção 1 ou 4, para que o código de *Huffman* já tenha sido executado e as informações necessárias estejam ordenadas. Além disso, é importante avisar que a interface busca auxiliar o usuário ao longo de todo o processo de compressão e descompressão através de instruções de entrada, como quando a extensão do arquivo deve ou não ser inserida. A extensão escolhida para os arquivos compactados foi a **".jlv"**, referente às iniciais dos nomes de cada membro do grupo, sendo esta a extensão gerada como saída na compressão e exigida como entrada na descompressão. O *Makefile* foi implementado visando otimizar o processo de compilar o código e fazer as verificações de vazamento de memória. O material utilizado como base foi de um trabalho prévio realizado em outra disciplina.

2 FILA DE PRIORIDADES PARA TIPOS GENÉRICOS DE DADOS

A seguir, a struct criada em **heap.h**:

Figura 1 - Representação da estrutura do Heap

```
typedef struct heap {  
    void **dados; //Fila de prioridade  
    int tamanho;  
    int capacidade;  
} Heap;
```

A estrutura *Heap* possui a variável *void** dados*, que atua como um vetor de ponteiros genéricos, ou seja, um apontamento para um vetor do tipo *void**. Com isso, podemos tratar tipos genéricos de dados, pois cada posição do vetor armazena o endereço para um dado qualquer. No nosso trabalho, focaremos no tipo NoHuffman, uma estrutura que será discutida posteriormente.

No arquivo **heap.c**, o parâmetro `bool (*ehMenor) (void*, void*)` é uma função de *callback* para tratarmos os diferentes tipos de dados. Para isso, foi necessário que seus parâmetros tivessem o tipo *void**. Com isso, podemos utilizá-lo para heaps de inteiros, caracteres, estruturas, etc. Vale ressaltar que, para cada tipo de dado, é necessário implementar novas funções de comparação específicas para a manipulação exigida no *heap*.

2.1 Funções complementares implementadas em heap.c

Nesta seção, serão expostas as funções auxiliares que têm como objetivo dar suporte às operações principais do Heap, sendo utilizadas internamente pela estrutura. São elas a função `troca()`, responsável pela permutação de elementos, e a função `mallocSafe()`, responsável pela alocação segura de memória, ambas fundamentais para o correto funcionamento das demais operações implementadas.

2.1.1 mallocSafe()

Figura 2 - Função mallocSafe() implementada em heap.c

```
void* mallocSafe(size_t nbytes) {  
    void *ptr = malloc(nbytes);  
    if (ptr == NULL) {  
        printf("Erro na alocação de memória\n");  
        exit(1);  
    }  
    return ptr;  
}
```

A função `mallocSafe()` é utilizada para tratar os casos de alocação dinâmica de memória de forma segura. Ela tem como parâmetro a quantidade de *bytes* a serem alocados, do tipo `size_t`, e tem como objetivo realizar a alocação de memória verificando se o procedimento foi executado com sucesso. Para isso, a função utiliza `malloc()` para reservar a quantidade de memória indicada em um ponteiro do tipo `void` e, em seguida, verifica se a alocação ocorreu corretamente. Ao final, retorna um ponteiro para a região de memória alocada, útil para o programa fazer o devido uso. Além disso, a função traz a segurança de que toda alocação dinâmica realizada emitirá um aviso no terminal em caso de erro na alocação de memória.

2.1.2 troca()

Figura 3 - Função troca() implementada em heap.c

```
void troca(void **x, void **y) {  
    void* temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

A função `troca()` é utilizada para realizar a troca de endereços de memória entre ponteiros do tipo `void*`, sendo chamada nas operações de ordenação na fila de prioridade para trocar elementos. Ela tem como parâmetro dois ponteiros do tipo `void` e realiza a troca entre ambos utilizando um ponteiro auxiliar. A função não possui retorno.

2.2 Funções principais implementadas em heap.c

Nesta seção, serão apresentadas as principais funções implementadas no arquivo **heap.c** responsáveis pelas operações básicas da estrutura *Heap*.

2.2.1 criaHeap()

Figura 4 - Função criaHeap() implementada em heap.c

```
void criaHeap(Heap* h, int capacidade) {  
    h->tamanho = 0;  
    h->capacidade = capacidade + 1; //Heap inicia da posição 1  
    h->dados = (void**) mallocSafe(h->capacidade * sizeof(void*));  
}
```

A função `criaHeap()` é utilizada para criar e inicializar a estrutura do Heap, sendo chamada para organizar a lógica de construção da árvore de *Huffman*. Ela tem os seguintes parâmetros: um ponteiro para o *Heap* e um inteiro representando a capacidade máxima. Sua operação consiste em definir o tamanho inicial como 0, a capacidade, passada como argumento acrescida de 1 para ignorar o índice 0, e acionar a função `mallocSafe()` para alocação do vetor genérico de dados. A função não possui retorno.

2.2.2 heapifyUp()

Figura 5 - Função heapifyUp() implementada em heap.c

```
void heapifyUp(Heap* h, int i, bool (*ehMenor)(void*, void*)) {  
    int pai = i / 2;  
    if (i > 1 && ehMenor(h->dados[i], h->dados[pai])) {  
        troca(&h->dados[i], &h->dados[pai]);  
        heapifyUp(h, pai, ehMenor);  
    }  
}
```

A função `heapifyUp()` é utilizada para manter a propriedade estrutural do Min-Heap subindo um elemento que acabou de ser inserido, sendo chamada na função de inserção para garantir que a raiz seja o elemento de menor frequência. Ela tem os seguintes parâmetros: um ponteiro para o *Heap*, um inteiro

representando o índice do elemento atual e uma função *Callback* de comparação (`ehMenor()`). Sua operação consiste em identificar se o elemento atual é menor que seu pai ($i / 2$), acionando a função `troca()` no caso da condição ser verdadeira. Assim, o código segue recursivamente para o novo índice do pai, subindo até organizar todo o *Min-heap*. A função não possui retorno.

2.2.3 heapifyDown()

Figura 6 - Função `heapifyDown()` implementada em `heap.c`

```
void heapifyDown(Heap* h, int i, bool (*ehMenor)(void*, void*)) {
    int esq = 2 * i;
    int dir = 2 * i + 1;
    int menor = i;

    if (esq <= h->tamanho && ehMenor(h->dados[esq], h->dados[menor]))
        menor = esq;

    if (dir <= h->tamanho && ehMenor(h->dados[dir], h->dados[menor]))
        menor = dir;

    if (menor != i) {
        troca(&h->dados[menor], &h->dados[i]);
        heapifyDown(h, menor, ehMenor);
    }
}
```

A função `heapifyDown()` é utilizada para manter a propriedade estrutural do Min-Heap descendo um elemento, sendo chamada após a remoção do elemento mínimo (raiz), organizando a fila. Ela tem os seguintes parâmetros: um ponteiro para o *Heap*, um inteiro representando o índice do elemento atual e uma função *Callback* de comparação (`ehMenor()`). Sua operação consiste em identificar se o elemento atual, substituto da raiz, é maior que um de seus filhos, esquerdo ($2 * i$) e direito ($2 * i + 1$), acionando a função `troca()` no caso da condição ser verdadeira. Assim, o código segue recursivamente para o menor filho, garantindo a ordem do *Min-heap*. A função não possui retorno.

2.2.4 insereHeap()

Figura 7 - Função insereHeap() implementada em heap.c

```
void insereHeap(Heap* h, void* dado, bool (*ehMenor)(void*, void*)) {
    if (h == NULL || dado == NULL) {
        printf("Erro! Heap ou dado não existente...\n");
        return;
    }

    if (h->tamanho == h->capacidade) {
        printf("Heap cheio!\n");
        return;
    }

    h->dados[h->tamanho + 1] = dado;
    h->tamanho++;

    if (h->tamanho > 1)
        heapifyUp(h, h->tamanho, ehMenor);
}
```

A função `insereHeap()` é utilizada para adicionar um novo elemento na fila de prioridade, sendo chamada para inserir os nós que formarão a árvore de Huffman. Ela tem os seguintes parâmetros: um ponteiro para o *Heap*, um ponteiro do tipo *void* para o elemento que será inserido e uma função *Callback* de comparação (`ehMenor()`). Sua operação consiste em inserir o elemento no final do *array*, incrementar o tamanho e acionar o `heapifyUp()` para reordenar. A operação ocorre apenas se houver espaço, abortando o procedimento e emitindo aviso caso a fila esteja lotada ou se o *heap*/dado não existir. A função não possui retorno.

2.2.5 extraiMin()

Figura 8 - Função extraiMin() implementada em heap.c

```
void* extraiMin(Heap* h, bool (*ehMenor)(void*, void*)) {
    if (h != NULL && h->tamanho > 0) {
        void* raiz = h->dados[1];
        h->dados[1] = h->dados[h->tamanho];
        h->tamanho--;
        heapifyDown(h, 1, ehMenor);
        return raiz;
    }
    return NULL;
}
```

A função `extraMin()` é utilizada para remover e retornar o elemento de maior prioridade (menor frequência) da fila, sendo chamada no processo de construção da árvore de Huffman. Ela tem como parâmetros: um ponteiro para o *Heap* e uma função *Callback* de comparação (`ehMenor()`). Sua operação consiste em guardar o elemento da raiz, colocar o último elemento como raiz e decrementar o tamanho. Por fim, aciona a função `heapifyDown()` para reorganizar o *Min-Heap*. A operação só ocorre se o heap existir e possuir elementos, abortando o procedimento e retornando `NULL`. A função retorna um ponteiro genérico para o elemento removido (menor frequência) que será reinserido na árvore.

2.2.6 liberaHeap()

Figura 9 - Função `liberaHeap()` implementada em `heap.c`

```
void liberaHeap(Heap* h) {
    if (h->dados != NULL) free(h->dados);
    if (h != NULL) free(h);
}
```

A função `liberaHeap()` é utilizada para deslocar a memória ocupada pela fila de prioridade, sendo chamada após a conclusão da montagem da árvore de Huffman. Ela tem como parâmetro um ponteiro para o *Heap*. Sua operação consiste em liberar o vetor de dados genéricos e ponteiro para a estrutura do *heap*. A operação só ocorre se o heap e o vetor de dados existirem, abortando o procedimento em caso `NULL`. A função não possui retorno.

3 IMPLEMENTAÇÃO DA COMPACTAÇÃO DE ARQUIVOS DE TEXTO

A seguir, a struct criada em `huffman.h`:

Figura 10 - Estrutura `NoHuffman` implementada em `Huffman.c`

```
typedef struct no {
    unsigned char chave;
    int frequencia;
    struct no* esq;
    struct no* dir;
} NoHuffman;
```

A estrutura `NoHuffman` representa um nó utilizado na construção da árvore de *Huffman*. Ela possui a variável chave do tipo *unsigned char*, responsável por armazenar o caractere associado ao nó, e a variável frequência do tipo inteiro, que armazena a quantidade de ocorrências desse caractere no arquivo analisado. Além disso, a estrutura contém os ponteiros *struct* `no*` `esq` e *struct* `no*` `dir`, que apontam, respectivamente, para os filhos à esquerda e à direita do nó na árvore. Dessa forma, cada elemento da árvore consegue armazenar tanto os dados necessários para o processo de compressão quanto as ligações entre os demais nós. No contexto do algoritmo de *Huffman*, os nós folha representam caracteres do arquivo original, enquanto os nós internos armazenam apenas a soma das frequências de seus filhos, permitindo a construção hierárquica da árvore de codificação.

3.1 Funções complementares implementadas em `huffman.c`

3.1.1 `ehMenorNo()`

Figura 11 - Função `ehMenorNo()` implementada em `huffman.c`

```
//Callback para o heap genérico
bool ehMenorNo(void* a, void* b) {
    NoHuffman* noA = (NoHuffman*)a;
    NoHuffman* noB = (NoHuffman*)b;
    return noA->frequencia < noB->frequencia;
}
```

A função `ehMenorNo()` é utilizada com função *Callback* do heap genérico implementado no sistema, sendo passada como argumento para as funções de gerenciamento do *Heap* com diferentes tipos de dados. Ela tem como parâmetro dois ponteiros do tipo *void* para os nós a serem comparados. Sua operação consiste em converter os ponteiros para o tipo *NoHuffman**, possibilitando o acesso às informações armazenadas nos nós da árvore de *Huffman*. Por fim, a função compara os valores das frequências dos dois nós e retorna um valor *bool*, sendo *true* caso o ponteiro *a* seja menor que o ponteiro *b*, ou *false* caso contrário. Assim, o *heap* mantém a propriedade de ordenação necessária para a construção da árvore de *Huffman*.

3.1.2 verificaArquivo()

Figura 12 - Função verificaArquivo() implementada em huffman.c

```
void verificaArquivo(FILE* arquivo) {  
    if (arquivo == NULL) {  
        printf("Erro ao abrir o arquivo!\n");  
        exit(1);  
    }  
}
```

A função `verificaArquivo()` é utilizada para validar se a abertura de um arquivo foi realizada corretamente, sendo acionada após as chamadas da função `fopen()`. Ela tem como parâmetro um ponteiro do tipo *FILE* correspondente ao arquivo. Sua operação consiste em verificar se o ponteiro recebido possui valor *NULL*. Caso isso ocorra, a função imprime a mensagem de erro e encerra o programa. Dessa forma, centraliza-se o tratamento de erros relacionados à manipulação de arquivos. A função não possui retorno.

3.1.3 formataNome()

Figura 13 - Função formataNome() implementada em huffman.c

```
void formataNome(char* nomeArquivo) {  
    int i = 0;  
    while (nomeArquivo[i] != '\n') {  
        i++;  
    }  
    nomeArquivo[i] = '\0';  
}
```

A função `formataNome()` é utilizada para remover o caractere de quebra de linha resultante do console, sendo chamada para limpar as entradas do usuário. Ela tem como parâmetro um ponteiro do tipo *char* correspondente ao nome, que será formatado, do arquivo. Sua operação consiste em percorrer a string caractere por caractere até encontrar o `'\n'`, substituindo-o pelo terminal `'\0'`. A função não possui retorno.

3.1.4 criaArquivoSaida()

Figura 14 - Função criaArquivoSaida() implementada em huffman.c

```
FILE* criaArquivoSaida(char* nomeArquivoSaida) {  
    // Cria o arquivo para escrita binaria e retorna aberto  
    FILE* arquivoSaida = fopen(nomeArquivoSaida, "wb");  
    verificaArquivo(arquivoSaida);  
    return arquivoSaida;  
}
```

A função `criaArquivoSaida()` é utilizada para abrir o arquivo onde será gravado o conteúdo compactado. É chamada no início do processo de compressão do arquivo. Ela tem como parâmetro um ponteiro do tipo *char* correspondente ao nome do arquivo que será aberto. Sua operação consiste em abrir o arquivo em modo de escrita binária, por meio da função `fopen()`, confirmando a abertura acionando a `verificaArquivo()`. A função retorna um ponteiro do tipo *FILE*, sendo utilizado para acessar o arquivo.

3.2 Funções principais implementadas em huffman.c

3.2.1 contaFrequencia()

Figura 15 - Função contaFrequencia() implementada em huffman.c

```
void contaFrequencia(FILE* arquivoLeitura, int* vetorFrequencia) {  
    int caractere;  
  
    // Adiciona-se 1 à posição referente ao caractere no vetor de Frequências:  
    while ((caractere = fgetc(arquivoLeitura)) != EOF) {  
        vetorFrequencia[(unsigned char)caractere]++;  
    }  
}
```

A função `contaFrequencia()` é utilizada para contagem de ocorrências de cada caractere no arquivo de entrada. É chamada no início do processo de compressão do arquivo. Ela possui os seguintes parâmetros: um ponteiro do tipo *FILE*, referente ao arquivo a ser lido, e um ponteiro do tipo *int*, que será um vetor

para armazenar as frequências. Sua operação consiste em percorrer todo o arquivo de leitura e, através da função `fgetc()`, ler todos os caracteres e incrementar a ocorrência em cada posição do vetor com o valor do caractere correspondente na tabela ASCII. A função não possui retorno.

3.2.2 `imprimeFrequencia()`

Figura 16 - Função `imprimeFrequencia()` implementada em `huffman.c`

```
void imprimeFrequencia(int vetorFrequencia[256]) {
    for (int i = 0; i < 256; i++) {
        if (vetorFrequencia[i] > 0) {
            if (i == '\n') {
                printf("\tASCII: %d, Caractere: \\n, Frequencia: %d\\n", i, vetorFrequencia[i]);
            }
            else if (i == '\t') {
                printf("\tASCII: %d, Caractere: \\t, Frequencia: %d\\n", i, vetorFrequencia[i]);
            }
            else if (i == '\r') {
                printf("\tASCII: %d, Caractere: \\r, Frequencia: %d\\n", i, vetorFrequencia[i]);
            }
            else if (i == ' ') {
                printf("\tASCII: %d, Caractere: espaco, Frequencia: %d\\n", i, vetorFrequencia[i]);
            }
            else {
                printf("\tASCII: %d, Caractere: %c, Frequencia: %d\\n", i, i, vetorFrequencia[i]);
            }
        }
    }
}
```

A função `imprimeFrequencia()` é utilizada para imprimir na tela o número de ocorrências não nulas de cada caractere do vetor de frequências. É chamada na função `main`. Ela possui como parâmetro um vetor de inteiro referente às ocorrências de cada caractere no arquivo de leitura. Sua operação consiste em percorrer todo vetor frequência e, caso possua o valor maior que zero, imprimir o código na tabela ASCII, o caractere em si e a frequência relativa. Vale ressaltar que os caracteres `'\n'`, `'\t'`, `'\r'` e `'espaco'` são caracteres que não são visíveis ou prejudicam a impressão, por isso há a verificação e caso seja um desses caracteres, o programa força a impressão. A função não possui retorno.

3.2.3 criaNoHuffman()

Figura 17 - Função criaNoHuffman() implementada em huffman.c

```
NoHuffman* criaNoHuffman(int chave, int frequencia) {
    NoHuffman* novoNo = mallocSafe(sizeof(NoHuffman));

    novoNo->chave = (unsigned char)chave;
    novoNo->frequencia = frequencia;
    novoNo->esq = NULL;
    novoNo->dir = NULL;

    return novoNo;
}
```

A função `criaNoHuffman()` é utilizada para criar um novo nó para a árvore de *Huffman*. É chamada na função `constroiArvore()`. Ela possui como parâmetro um inteiro representando a chave armazenada no novo nó e a frequência do mesmo. Sua operação consiste em chamar a função `mallocSafe()` e preencher os campos de chave e frequência com os valores passados como argumento e definir o ponteiro para a esquerda e direita como *NULL*, pois o nó será adicionado como folha. A função não possui retorno.

3.2.4 constroiArvore()

Figura 18 - Função constroiArvore() implementada em huffman.c

```
NoHuffman* constroiArvore(int vetorFrequencia[256]) {
    Heap* h = mallocSafe(sizeof(Heap));
    criaHeap(h, 256);

    for (int i = 0; i < 256; i++) {
        if (vetorFrequencia[i] > 0) {
            NoHuffman* novoNo = criaNoHuffman(i, vetorFrequencia[i]);
            insereHeap(h, novoNo, ehMenorNo);
        }
    }

    while (h->tamanho > 1) {
        NoHuffman* menor1 = (NoHuffman*) extraiMin(h, ehMenorNo);
        NoHuffman* menor2 = (NoHuffman*) extraiMin(h, ehMenorNo);
        NoHuffman* novoNo = criaNoHuffman('\0', menor1->frequencia + menor2->frequencia);
        novoNo->esq = menor1;
        novoNo->dir = menor2;
        insereHeap(h, novoNo, ehMenorNo);
    }

    NoHuffman* raiz = (NoHuffman*) extraiMin(h, ehMenorNo);
    liberaHeap(h);
    return raiz;
}
```

A função `constroiArvore()` é utilizada para gerar a árvore de *Huffman* de acordo com as frequências de cada caractere. É chamada na função de compressão, descompressão e na main. Possui como parâmetro o vetor inteiro de frequências. Sua operação consiste em percorrer o vetor e, para cada posição com valor maior que um, cria-se um novo nó através da função `criaNoHuffman()` e logo em seguida adiciona no *heap*. Após a criação do *Min-Heap*, enquanto tiver elemento na estrutura, retira-se os dois menores elementos através da função `extraiMin()` e chama a função `insereHeap()`, adiciona-se o nó com a frequência resultante da soma dos dois menores nós e define os filhos como os nós retirados anteriormente. Com isso, a raiz é definida através da chamada da `extraiMin()` após toda a criação da árvore, libera-se o *heap* e retorna a raiz. O retorno da raiz será importante para manter o acesso à árvore e para controle na main.

3.2.5 criaTabelaCodigos()

Figura 19 - Função `criaTabelaCodigos()` implementada em `huffman.c`

```
void criaTabelaCodigos(NoHuffman* raiz, int profundidade, char* codigo, unsigned char tabela[256][256]){
    if (raiz == NULL) return;

    // Encontrou o no folha
    if (raiz->dir == NULL && raiz->esq == NULL) {
        // Preenche todo o vetor correspondente a posicao do caractere com seu devido codigo
        for (int i = 0; i <= profundidade; i++) {
            tabela[raiz->chave][i] = codigo[i];
        }
        tabela[raiz->chave][profundidade] = '\0';
    }

    if (raiz->esq != NULL) {
        codigo[profundidade] = '0';
        criaTabelaCodigos(raiz->esq, profundidade + 1, codigo, tabela);
    }

    if (raiz->dir != NULL) {
        codigo[profundidade] = '1';
        criaTabelaCodigos(raiz->dir, profundidade + 1, codigo, tabela);
    }
}
```

A função `criaTabelaCodigos()` é utilizada para gerar os códigos binários para cada caractere. É chamada no processo de compressão para criar um dicionário, sendo consultada durante a codificação. Ela possui os seguintes

parâmetros: um ponteiro do tipo *NoHuffman*, referente ao nó atual (ou raiz) da árvore, um inteiro, para profundidade na árvore, um ponteiro do tipo *char*, que será um vetor para armazenar os códigos durante a construção, e uma matriz de *unsigned char* que será o dicionário. Sua operação consiste em preencher a matriz com os códigos de cada caractere de forma recursiva, percorrendo a árvore em pré-ordem. Primeiro, ela checa se o nó atual não tem filhos, o que significa que encontrou um caractere. Se isso for verdade, o código gerado até o momento é salvo na linha da matriz desse caractere e encerrado com '\0'. Caso contrário, ela avalia as ramificações, colocando '1' no vetor se houver filho à direita, ou '0' se houver à esquerda, e chama a si mesma novamente para continuar a codificação. O processo ocorre apenas se o ponteiro para a raiz existir. A função não possui retorno.

3.2.6 imprimeArvore()

Figura 20 - Função `imprimeArvore()` implementada em `huffman.c`

```
void imprimeArvore(NoHuffman* raiz, int profundidade) {
    if (raiz == NULL) return;

    imprimeArvore(raiz->dir, profundidade + 1);

    for (int i = 0; i < profundidade; i++) {
        printf("        ");
    }

    if (raiz->esq == NULL && raiz->dir == NULL) {
        if (raiz->chave == '\n') {
            printf("[\\n : %d]\\n", raiz->frequencia);
        }
        else if (raiz->chave == '\t') {
            printf("[\\t : %d]\\n", raiz->frequencia);
        }
        else if (raiz->chave == '\r') {
            printf("[\\r : %d]\\n", raiz->frequencia);
        }
        else if (raiz->chave == ' ') {
            printf("[espaco : %d]\\n", raiz->frequencia);
        }
        else {
            printf("[%c : %d]\\n", raiz->chave, raiz->frequencia);
        }
    }

    else {
        printf("[%d]\\n", raiz->frequencia);
    }

    imprimeArvore(raiz->esq, profundidade + 1);
}
```

A função `imprimeArvore()` é utilizada para impressão da árvore de *Huffman* com os caracteres e as suas respectivas frequências. É chamada na função `main`. Ela possui os seguintes parâmetros: um ponteiro do tipo *NoHuffman* para a raiz e um inteiro referente a profundidade de cada nó na árvore. Sua operação consiste em percorrer a árvore, em ordem, para impressão de acordo com o nível, para que tenha simetria, utiliza-se a profundidade para impressão de espaços e, caso esteja em um nó folha, imprime-se o caractere e a frequência relativa. Pelo mesmo motivo que ocorre na função `imprimeFrequencia()`, há o tratamento na impressão para os caracteres especiais, sendo eles `'\n'`, `'\t'`, `'\r'`, `'espaço'`. A função não possui retorno.

3.2.7 `comprimeArquivo()`

Primeiramente, vale ressaltar que a implementação da função de compressão possui um grande quantidade de linhas, pensando nisso, a explicação da função será feita por partes visando melhor entendimento.

A função `comprimeArquivo()` é utilizada para compactação do conteúdo pertencente ao arquivo de leitura. É chamada na função `main`. Ela possui os seguintes parâmetros: dois ponteiros do tipo *char*, o primeiro referente ao nome do arquivo de leitura e o segundo referente ao arquivo de saída e um vetor de inteiro com as frequências relativas de cada caractere.

Figura 21 - Função `comprimeArquivo()` implementada em `huffman.c`

```
NoHuffman* comprimeArquivo(char* nomeArquivoLeitura, char* nomeArquivoSaida, int vetorFrequencia[256]) {
    FILE* arquivoLeitura = fopen(nomeArquivoLeitura, "r");
    verificaArquivo(arquivoLeitura);
    contaFrequencia(arquivoLeitura, vetorFrequencia);

    NoHuffman* raiz = constroiArvore(vetorFrequencia);
    unsigned char tabelaCodigos[256][256];
    char codigo[256];

    memset(tabelaCodigos, 0, sizeof(tabelaCodigos));

    criaTabelaCodigos(raiz, 0, codigo, tabelaCodigos);
}
```

A operação acima consiste em abrir o arquivo de leitura com o nome passado como argumento, logo após, ocorre a verificação para garantir que o arquivo correto seja aberto, e há a chamada da função `contaFrequencia()`. Com isso, chama-se a função `constroiArvore()`, atribui-se o resultado à raiz e declara a tabela de

códigos e o vetor que será impresso no cabeçalho do arquivo compactado. Por último, cria-se a tabela de código e inicia com '\0' através da função `memset()`. Por fim, preenche-se a tabela de códigos através da chamada da função `criaTabelaCodigos()`.

Figura 22 - Continuação da função `comprimeArquivo()`

```
FILE* arquivoSaida = criaArquivoSaida(nomeArquivoSaida);

// Escreve cabecalho no arquivo de saida
fwrite(vetorFrequencia, sizeof(int), 256, arquivoSaida);

// Retorna o ponteiro para o inicio do arquivo
rewind(arquivoLeitura);

int c, contadorBits = 0;
unsigned char aux = 0;
while ((c = fgetc(arquivoLeitura)) != EOF) {

    // Representacao do caractere em bits
    int tamanho = strlen((char*) tabelaCodigos[c]);
    for (int i = 0; i < tamanho; i++) {

        // Transforma o codigo do caractere em inteiro 0 e 1
        int bit = tabelaCodigos[c][i] - '0';

        // Move o auxiliar para esquerda e adiciona o bit referente ao codigo do caractere
        aux = (aux << 1) | bit;
        contadorBits++;

        if (contadorBits == 8) {
            fputc(aux, arquivoSaida);
            aux = 0;
            contadorBits = 0;
        }
    }
}
```

Após a etapa inicial, cria-se o arquivo de saída através da função `criaArquivoSaida()`. Com o arquivo criado, o vetor de frequências é escrito no cabeçalho do arquivo de saída, necessário para a futura descompressão, e retorna o ponteiro para o início do arquivo com a função `rewind()`. A partir do cabeçalho escrito, declara-se as variáveis auxiliares e inicia a leitura dos caracteres do arquivo de entrada através da iteração da função `fgetc()` até o final do arquivo. A variável `tamanho` armazena o comprimento do código referente ao caractere e itera em relação a esse valor. Dessa forma, há a conversão do caractere para o tipo inteiro e atribui na variável `bit` através da subtração pelo caractere '0', o `aux`, variável do tipo

unsigned char declarada em cima, funcionará como um “vetor de bits” armazenando os códigos e, através da manipulação de bit, move-se um bit à esquerda através do operador `<<`, faz a operação de “ou” através do operador `|` e acrescenta um na variável do tipo inteiro auxiliar `contadorBits`. Após a manipulação, verifica-se se a quantidade de *bits* é igual a 8, caso seja, significa que o *byte* pode ser escrito no arquivo de saída e isso acontece através da função `fputc()`, depois zera tanto o `aux` quanto o `contadorBits` e reinicia todo o processo para o restante dos binários.

Figura 23 - Final da função `comprimeArquivo()`

```
// Caso tenha sobrado algum bit
if (contadorBits > 0) {
    aux = aux << (8 - contadorBits);
    fputc(aux, arquivoSaida);
}

fclose(arquivoLeitura);
fclose(arquivoSaida);
return raiz;
}
```

Por conseguinte, há a verificação para caso tenha sobrado uma quantidade de bits superior a 0, caso tenha, completa-se o `aux` com zeros e coloca o último byte no arquivo de saída. Para finalizar, fecha-se ambos os arquivos e retorna a raiz. Esse retorno será útil para acessar a árvore de *Huffman* e controle na função `main`.

3.2.8 `descomprimeArquivo()`

Assim como na função de compressão, a implementação da função de descompressão possui um grande quantidade de linhas, a explicação está disposta em partes visando melhor entendimento.

A função `descomprimeArquivo()` é utilizada para descompactação do conteúdo pertencente ao arquivo de leitura comprimido. É chamada na função `main`. Ela possui os seguintes parâmetros: dois ponteiros do tipo *char*, o primeiro referente ao nome do arquivo de leitura e o segundo referente ao arquivo de saída.

Figura 24 - Função descomprimeArquivo() implementada em huffman.c

```

NoHuffman* descomprimeArquivo(char* nomeArquivoLeitura, char* nomeArquivoSaida, int vetorFrequencia[256]) {

    FILE* arquivoLeitura = fopen(nomeArquivoLeitura, "rb");
    verificaArquivo(arquivoLeitura);

    fread(&vetorFrequencia, sizeof(int), 256, arquivoLeitura);
    NoHuffman* raiz = constroiArvore(vetorFrequencia);

    unsigned char tabelaCodigos[256][256];
    char codigo[256];
    memset(tabelaCodigos, 0, sizeof(tabelaCodigos));
    criaTabelaCodigos(raiz, 0, codigo, tabelaCodigos);

    FILE* arquivoSaida = fopen(nomeArquivoSaida, "w");
    verificaArquivo(arquivoSaida);
}

```

A operação acima consiste em abrir o arquivo de leitura com o nome passado como argumento, realizando a verificação pós-abertura para garantir que o arquivo foi aberto corretamente. Na sequência, há uma declaração de um vetor estático de inteiros de tamanho 256, representando os caracteres presentes na tabela ASCII, para guardar o vetor de frequência presente no cabeçalho do arquivo, sendo esse lido pela função `fread()`. Adiante, chama-se a função `constroiArvore()`, para o vetor de frequência, e atribui-se o resultado à raiz, refazendo a árvore para decodificação do arquivo. Ademais, ocorre a abertura do arquivo de saída, seguindo o nome passado como argumento e verificação de sucesso.

Figura 25 - Continuação da função descomprimeArquivo()

```

int c;
NoHuffman* atual = raiz;

unsigned char aux;
int contadorBits;
int caracteresEscritos = 0;

while((c = fgetc(arquivoLeitura)) != EOF && caracteresEscritos < raiz->frequencia) {
    aux = 0;
    contadorBits = 0;

    while (contadorBits < 8 && caracteresEscritos < raiz->frequencia) {
        //Caso a raiz seja folha, ou seja, apenas um caractere no txt original:
        if (raiz->esq == NULL && raiz->dir == NULL) {
            fputc(atual->chave, arquivoSaida);
            caracteresEscritos++;
            contadorBits++;
        }
    }
}

```

Dando continuidade, a operação acima consiste em preparar as variáveis que serão utilizadas na decodificação do arquivo como: 'c', um inteiro para guardar os

caracteres binários; 'atual', um ponteiro do tipo *NoHuffman* para percorrer a árvore de *Huffman*; 'aux', um *unsigned char* que atua similar a um vetor de *bits* para descompactação; 'contadorBits', um inteiro para contar quantos *bits* foram inseridos em 'aux'; 'caracteresEscritos', um inteiro para contar quantos caracteres foram decodificados durante a execução do processo.

Nesse sentido, é iniciada a leitura dos caracteres do arquivo de entrada através da iteração da função `fgetc()` até o final do arquivo e enquanto a quantidade de caracteres descompactadas for menor que a quantidade total de caracteres. Em sequência, ocorre a iteração responsável por preencher o “vetor de *bits*” e imprimir os caracteres originais no arquivo de saída, com a função `fputc()`. Assim, esse acontece enquanto 'contadorBits' for menor que oito, ou seja, até completar um *byte*. Por conseguinte, a operação inicia verificando se o nó atual é uma folha, imprimindo o caractere e incrementando a quantidade de caracteres escritos e o contador de *bits*.

Figura 26 - Final da descomprimeArquivo()

```

else {
    aux = (c >> (7 - contadorBits)) & 1;
    contadorBits++;

    if (aux == 0) atual = atual->esq;
    else atual = atual->dir;

    if (atual->dir == NULL && atual->esq == NULL) {
        fputc(atual->chave, arquivoSaida);
        atual = raiz;
        caracteresEscritos++;
    }
}

fclose(arquivoLeitura);
fclose(arquivoSaida);

return raiz;
}

```

Ainda nesse contexto, caso a condição anterior seja falsa, a função inicia preenchendo 'aux', através da manipulação de *bit* a *bit*. Assim, move-se uma quantidade de *bits*, resultando da diferença de 7 e 'contadorBits', à direita por meio

do operador `>>` e realiza a operação de “e” através do operador “&” com o inteiro ‘1’, aumentando o contador de bits. Na sequência, a árvore é percorrida seguindo o valor das arestas. Por fim, uma nova verificação de folha é feita após a travessia na árvore, imprimindo o caractere no arquivo de saída, usando `fputc()`, atribuindo o nó atual à raiz e incrementando os caracteres escritos.

Para finalizar a função, fecha-se ambos os arquivos e retorna a raiz. Esse retorno será útil para acessar a árvore de *Huffman* e controle na função `main`.

3.2.9 liberaArvore()

Figura 27 - Função `liberaArvore()` implementada em `huffman.c`

```
void liberaArvore(NoHuffman* raiz) {  
    if (raiz == NULL) return;  
  
    liberaArvore(raiz->esq);  
    liberaArvore(raiz->dir);  
    free(raiz);  
}
```

A função `liberaArvore()` é utilizada para liberar recursivamente toda a memória alocada dinamicamente para os nós da árvore, sendo chamada, caso seja necessário, antes de realizar as operações de compressão/descompressão no menu (opção 1 e 4) e após todas as operações realizadas (opção 5). Ela possui como parâmetro um ponteiro do tipo *NoHuffman* para a raiz da árvore que será liberada. Sua operação consiste em percorrer a árvore em pós ordem, chama a si mesma para limpar o filho esquerdo, depois limpa o filho direito e, por fim, desaloca o nó atual utilizando a função `free()`. A função não possui retorno.

4 MENU IMPLEMENTADO EM MAIN.C

O arquivo `main.c` é responsável pela interface entre o usuário e o sistema de compactação, sendo o ponto de entrada do programa. Por meio de um menu interativo, o usuário pode realizar as operações: compactação de um arquivo de

texto, exibição da tabela de frequências dos caracteres, impressão da árvore de Huffman gerada, e a descompactação de um arquivo previamente comprimido.

Figura 28 - Início da função main() implementada em main.c

```
int main (void) {
    int vetorFrequencia[256] = {0};
    int opcao, aux = 0;

    // criado para saber se alguma operacao ja criou uma arvore
    // setado para NULL antes de toda compressão ou descompressão, e passado como referência
    NoHuffman* raiz = NULL;
```

Ao iniciar a função `main()`, é declarado o vetor de inteiros `vetorFrequencia[]` com 256 posições, cada uma referente a um valor da tabela ASCII. Este vetor será passado como argumento para as funções `contaFrequencia()`, `imprimeFrequencia()`, `comprimeArquivo()`. Logo após, são declarados as variáveis do tipo inteiro `opção` e `aux` - este recebendo 0 desde o início - uma responsável por armazenar a operação escolhida pelo usuário e a outra por garantir que houve pelo menos uma compressão antes de o sistema exibir a tabela de códigos e a árvore de Huffman. Por fim, é declarada a variável `raiz`, ponteiro do tipo `NoHuffman` que guarda a raiz da última árvore criada (tanto na compressão quanto na descompressão).

Figura 29 - Loop do menu implementado

```
do {
    printf("\n ===== Menu de opcoes ===== \n");
    printf("1 - Comprimir um arquivo\n");
    printf("2 - Contagem de ocorrencias\n");
    printf("3 - Imprimir o Codigo de Huffman\n");
    printf("4 - Descomprimir um arquivo\n");
    printf("5 - Sair\n");
    printf("Escolha uma opcao: ");

    scanf("%d", &opcao);
    getchar(); // evita que Opcao receba \n
```

A execução se inicia apresentando no terminal o menu de opções e a solicitação de entrada do usuário. A entrada é armazenada em `opção`, cujo valor

aciona um dos casos da estrutura *switch*. Porém, se o valor não corresponder ao menu, uma notificação é acionada e um novo laço é executado, até que esse valor satisfaça os valores 1, 2, 3, 4 ou 5.

Figura 30 - Case 1 para compressão de arquivos

```
switch (opcao) {

    case 1:
        char nomeArquivoEntrada[100];
        char nomeArquivoSaida[100];

        if (raiz != NULL) liberaArvore(raiz);
        memset(vetorFrequencia, 0, sizeof(vetorFrequencia));

        printf("Digite o nome do arquivo de entrada [defina a extensao Ex.: .txt]: ");
        fgets(nomeArquivoEntrada, 100, stdin);
        printf("Digite o nome do arquivo de saida [sem extensao]: ");
        fgets(nomeArquivoSaida, 100, stdin);

        formataNome(nomeArquivoEntrada);
        formataNome(nomeArquivoSaida);

        strcat(nomeArquivoSaida, ".jlv");

        // Compara se os nomes serão iguais, para evitar erro
        while (strcmp(nomeArquivoEntrada, nomeArquivoSaida) == 0) {
            printf("O nome dos arquivos sao iguais, digite outro nome para o arquivo de saida [sem extensao]: ");
            fgets(nomeArquivoSaida, 100, stdin);
            formataNome(nomeArquivoSaida);
            strcat(nomeArquivoSaida, ".jlv");
        }

        //Inicia a compressão do arquivo
        raiz = comprimeArquivo(nomeArquivoEntrada, nomeArquivoSaida, vetorFrequencia);

        printf("\nCompressao concluida!\n");
        aux = 1;
        break;
}
```

Caso a variável *opcao* tenha o valor 1, o programa prepara-se para comprimir um arquivo previamente criado. Depois de criar os vetores estáticos para os nomes dos arquivos de entrada e saída, o sistema verifica se já existe uma árvore de huffman criada, que em caso positivo, libera essa árvore da memória antes da criação de uma nova ao comprimir. O sistema garante que o vetor de frequências esteja zerado, para que não haja nenhum problema de sobreposição ao realizar uma operação após a outra. O sistema lê os nomes dos arquivos de entrada e saída e os formata. A extensão *.jlv* é adicionada ao nome do arquivo de saída e os dois nomes são comparados a fim de garantir que não sejam iguais, evitando que o programa leia determinado arquivo ao mesmo tempo que o sobrescreve, o que geraria um erro.

Com isso, a variável *raiz* recebe a árvore de Huffman gerada ao comprimir o arquivo (valor retornado de `comprimeArquivo()`), *aux* recebe o valor 1, permitindo a exibição da árvore gerada, além da lista da frequência de cada caractere no arquivo. Para finalizar, caso a compressão tenha sido realizada com sucesso, imprime-se na tela uma mensagem de confirmação para o usuário.

Figura 31 - Case 2 para impressão da tabela de frequências

```
case 2:
    if (aux == 0) {
        printf("\nErro! Voce ainda nao compactou um arquivo\n");
        break;
    }

    // Caso ja tenha passado pela opcao 1, imprime as frequencias de cada caracter na tela
    printf("\n == Imprimindo Frequencias == \n");
    imprimeFrequencia(vetorFrequencia);
    break;
```

O caso correspondente à opção 2 somente é executado quando já houve uma compressão (*aux* diferente de 0). Assim é chamada a função `imprimeFrequencia()` passando `vetorFrequencia[]` como argumento.

Figura 32 - Case 3 para impressão da árvore

```
case 3:
    if (aux == 0) {
        printf("\nErro! Voce ainda nao compactou um arquivo\n");
        break;
    }

    printf("\n == Imprimindo Arvore de Huffman == \n");

    // Caso ja tenha passado pela opcao 1, imprime a arvore na tela
    imprimeArvore(raiz, 0);
    break;
```

O caso 3, assim como o caso 2 acima, só aciona a impressão da árvore de *Huffman* caso a compressão tenha acontecido pelo menos uma vez. Aciona a chamada para `imprimeArvore()`. Como nesse ponto, *aux* é diferente de 0, há garantia de que a raiz aponta para uma árvore válida, sendo a mesma passada como argumento da função.

Figuras 33.1 e 33.2 - Case 4 para descompressão de arquivos

```

case 4:
    if (raiz != NULL) liberaArvore(raiz);
    memset(vetorFrequencia, 0, sizeof(vetorFrequencia));

    char nomeArquivoComprimido[100];
    char nomeArquivoDescompactado[100];

    printf("Digite o nome do arquivo de entrada [.jlv]: ");
    fgets(nomeArquivoComprimido, 100, stdin);
    printf("Digite o nome do arquivo de saída [defina a extensão Ex.: .txt]: ");
    fgets(nomeArquivoDescompactado, 100, stdin);

    // Retira o \n dos 2 nomes e chama a descompressão
    formataNome(nomeArquivoComprimido);
    formataNome(nomeArquivoDescompactado);

    // Compara se os nomes serão iguais, para evitar erro
    while (strcmp(nomeArquivoComprimido, nomeArquivoDescompactado) == 0) {
        printf("O nome dos arquivos são iguais, digite outro nome para o arquivo de saída [sem extensão]: ");
        fgets(nomeArquivoSaida, 100, stdin);
        formataNome(nomeArquivoSaida);
        strcat(nomeArquivoSaida, ".jlv");
    }

    raiz = descomprimeArquivo(nomeArquivoComprimido, nomeArquivoDescompactado, vetorFrequencia);
    printf("\nDescompressão concluída!\n");
    aux = 1;
    break;

```

No caso da descompactação de um arquivo (opção 4), primeiramente, a árvore anterior é liberada e `vetorFrequencia[]` é redefinido para conter apenas 0, pois o processo de descompressão recria a árvore de Huffman a partir do cabeçalho do arquivo a descompactar. Assim como no primeiro caso, são criados e recolhidos os nomes dos arquivos de entrada e saída, e verificado se os nomes são iguais. Visto que os nomes não causam conflito, a raiz receberá o retorno da função `descomprimeArquivo()`, onde serão passados os nomes para os arquivos de entrada e saída. Por fim, `aux` é definido para 1, para que as opções 2 e 3 referentes ao arquivo descompactado sejam impressas no terminal.

Figura 34 - Case 5 para sair do programa

```

case 5:
    printf("\nEncerrando o programa!\n");
    liberaArvore(raiz);
    exit(0);

default:
    printf("\nOpção Inválida! Tente novamente\n");
    break;
}
} while (opcao != 5);

return 0;
}

```

Ao final do *switch*, temos a opção de saída do programa, que garante que a última árvore alocada seja liberada antes de finalizar a aplicação. A opção *default* apenas indica que o valor não corresponde às opções válidas, fazendo o programa retornar ao menu.

5 RESULTADOS E EXEMPLOS

Para executar o programa, utilize o comando *make run*, recomenda-se utilizar o *make clean* para excluir os arquivos “.o” ao término do programa.

Figura 35 - Comandos para executar o programa

```
ed2-compressaoDeArquivos$ make run && make clean
```

Após executar o comando, o programa será compilado e irá aparecer uma mensagem confirmando a compilação. Logo após será impresso na tela o menu de opções esperando que o usuário digite um número de 1 a 5.

Figura 36 - Representação do menu no terminal

```
* Compilando main.c
gcc -c -O0 -std=c11 -Wall -Werror -Wextra -Wno-sign-compare -Wno-unused-parameter -Wno-unused-variable -Wshadow main.c
* Compilando heap.c
gcc -c -O0 -std=c11 -Wall -Werror -Wextra -Wno-sign-compare -Wno-unused-parameter -Wno-unused-variable -Wshadow heap.c
* Compilando huffman.c
gcc -c -O0 -std=c11 -Wall -Werror -Wextra -Wno-sign-compare -Wno-unused-parameter -Wno-unused-variable -Wshadow huffman.c
Ligando objetos e gerando executável...
gcc main.o heap.o huffman.o -lm -o Compacta.exe
[OK] Compilação concluída.
Para executar o programa digite ./Compacta.exe no terminal.
Executando o programa:
./Compacta.exe

==-- Menu de opcoes ==--
1 - Comprimir um arquivo
2 - Contagem de ocorrencias
3 - Imprimir o Codigo de Huffman
4 - Descomprimir um arquivo
5 - Sair
Escolha uma opcao: |
```

Para exemplificar, o restante do texto irá demonstrar o passo a passo para comprimir, imprimir a tabela, imprimir a árvore e descomprimir um arquivo, nesse caso, o arquivo `asdfgh.txt`. Ao digitar 1, para comprimir um arquivo, devemos informar um arquivo de texto já existente na pasta para ser compactado. Logo depois informamos qual nome queremos que o arquivo compacto tenha (sem a extensão do arquivo).

Figura 37 - Representação do menu no terminal

```
Escolha uma opcao: 1
Digite o nome do arquivo de entrada [defina a extensao Ex.: .txt]: asdfgh.txt
Digite o nome do arquivo de saida [sem extensao]: asdfgh

Compressao concluida!
```

Após a compressão do arquivo, retorna-se para o menu e escolhe a opção 2 para impressão da árvore.

Figura 38 - Representação saída da opção 2 no terminal

```
Escolha uma opcao: 2

== Imprimindo Frequencias ==

ASCII: 97, Caractere: a, Frequencia: 5504
ASCII: 100, Caractere: d, Frequencia: 5120
ASCII: 102, Caractere: f, Frequencia: 5632
ASCII: 103, Caractere: g, Frequencia: 2944
ASCII: 104, Caractere: h, Frequencia: 2816
ASCII: 106, Caractere: j, Frequencia: 2688
ASCII: 115, Caractere: s, Frequencia: 5888
```

Ao escolher a opção 3, será impressa uma representação da árvore de *Huffman*, onde o valor mais à esquerda é o nó que representa o total de ocorrências, e todas as folhas são visíveis por conter, além de sua frequência, qual caractere representam.

Figura 39 - Representação saída da opção 3 no terminal

```
Escolha uma opcao: 3

== Imprimindo Arvore de Huffman ==

                        [h : 2816]
                        [5504]
                        [j : 2688]
                    [11008]
                [a : 5504]
            [19072]
                [d : 5120]
                [8064]
                [g : 2944]
        [30592]
                [s : 5888]
            [11520]
                [f : 5632]
```

Ao selecionar a opção 4, devemos informar o arquivo que será descompactado, além do nome que se deseja que o arquivo-resultado possua (este sendo preferível que se especifique a extensão).

Figura 40 - Representação da saída da opção 4 no terminal

```
Escolha uma opcao: 4
Digite o nome do arquivo de entrada [.jlv]: asdfgh.jlv
Digite o nome do arquivo de saida [defina a extensao Ex.: .txt]: asdfgh_desc.txt
Descompressao concluida!
```

Por fim, selecionando a opção 5, o sistema finaliza a operação e limpa os arquivos objetos, como comentado anteriormente

Figura 41 - Encerramento do programa

```
Escolha uma opcao: 5

Encerrando o programa!
🧹 Limpando arquivos compilados...
rm -rf *.o Compacta.exe
🌸 Diretório limpo!
```

Para verificar que a compactação e descompactação, pode-se utilizar o comando `diff -s nomeArquivo.txt nomeArquivo_saida.txt`. Caso apareça a mensagem que os arquivos são idênticos, significa que a operação foi realizada com sucesso.

Figura 42 - Comando para comparação de arquivos

```
diff -s asdfgh.txt asdfgh_saida.txt
Files asdfgh.txt and asdfgh_saida.txt are identical
```

Compactação dos arquivos de exemplo:

Arquivo contendo apenas 6 caracteres diferentes: `asdfgh.txt` (30.592 caracteres);

Figura 43 - Comparação dos arquivos no Explorador de Arquivos

 asdfgh.jlv	Arquivo JLV	12 KB
 asdfgh	Documento de Texto	30 KB

Arquivo com aproximadamente 40% do tamanho original, sendo a melhor compactação relativa dos 4 exemplos. Isso é devido à grande repetição de caracteres, mas com pouca variedade (apenas letras minúsculas de a até h), o que gera uma árvore menor, que possui menos algarismos para compactação, resultando

Arquivo com 100.000 caracteres com apenas letras maiúsculas e minúsculas do alfabeto: letras_100000.txt;

Figura 44 - Comparação dos arquivos no Explorador de Arquivos

 letras_100000.jlv	Arquivo JLV	72 KB
 letras_100000	Documento de Texto	98 KB

O arquivo teve uma compactação de aproximadamente 73% do tamanho original, e foi a pior das compressões. Isso ocorre devido à aleatoriedade dos caracteres, que não seguem uma estrutura de linguagem - onde existem símbolos mais comuns que outros - e o código de *Huffman* não demonstra eficiência, pois as reduções mais radicais que a sua árvore possibilita são menos utilizadas em comparação a um arquivo onde há uma linguagem definida.

Arquivo com 100.000 caracteres pertencentes a: alfabeto, acentuações, espaços, pontuações (ponto, vírgula, hífen, exclamação, etc), quebra de linha (“enter”) e TAB: latim_100000.txt;

Figura 45 - Comparação dos arquivos no Explorador de Arquivos

 latim_100000.jlv	Arquivo JLV	61 KB
 latim_100000	Documento de Texto	104 KB

Essa compressão foi de cerca de 59% do tamanho do arquivo original, e este aumento de rendimento é devido à estrutura do código, onde, por mais que sem semântica, o texto possui concordância léxica e sintática, ou seja, se assemelha a um texto comum no latim, com caracteres mais comuns que outros, alta ocorrência de espaços, várias quebras de linha e etc.

Arquivo com mais de 500.000 caracteres pertencentes a: alfabeto, acentuações, espaços, pontuações (ponto, vírgula, hífen, exclamação, etc), quebra de linha (“enter”) e TAB: biblia.txt (3.924.607 caracteres).

Figura 46 - Comparação dos arquivos no Explorador de Arquivos

 biblia.jlv	Arquivo JLV	2.279 KB
 biblia	Documento de Texto	3.961 KB

O arquivo após a compressão representa cerca de 59% do tamanho original, muito similar à compressão do arquivo com palavras em latim acima. Vale ainda ressaltar que a bíblia em questão está em português, cuja estrutura sintática é similar a do latim. Isso demonstra que o trunfo da compressão de Huffman está na recorrência de caracteres, já que arquivos com estruturas e palavras semelhantes tendem à mesma taxa de compressão.

É válido apontar que a quantidade de caracteres de cada arquivo foi informada pelo bloco de notas, que contabiliza apenas caracteres perceptíveis. Em um arquivo de texto, existem caracteres não perceptíveis - como exemplo, cada `\n` implica em um `\r` - logo o total de ocorrências identificadas no código tende a subir ao utilizar o código de Huffman e verificar o valor da raiz de sua árvore.

Os arquivos e seus respectivos compactos (.jlv) estão disponíveis nesta pasta:

<https://drive.google.com/drive/folders/1F0kH-laniu5nrC2RKmyYlO94Ujn30ema?usp=sharing>

REFERÊNCIAS

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Introduction to algorithms**. 3. ed. Cambridge: The MIT Press, 2009.

UNIVERSIDADE DE SÃO PAULO. Instituto de Matemática e Estatística. **Alocação dinâmica**. Disponível em: <http://www.ime.usp.br/~pf/algoritmos/aulas/aloca.html>. Acesso em: 24 maio 2026.

ZIVIANI, Nivio. **Projeto de algoritmos com implementações em Pascal e C**. 4. ed. São Paulo: Pioneira, 1999.